AirGrid Backend File Guide

What each backend file does, what it controls, and how frontend requests move through the system

1. Backend Big Picture

AirGrid backend is the control room behind the planner canvas. The frontend sends canvas data - area size, grid size, nodes, device properties, and visual settings - to the Express backend. The backend then analyzes layouts, optimizes them, and saves or loads project files as JSON.

Frontend Canvas	API Layer	Routes	Controllers	Engines	JSON Storage	Response

Flow: React canvas -> fetch API -> Express route -> controller -> engine/storage -> JSON response -> UI update

2. Folder Structure

```
server/
  server.js
  package.json
  .env
  config/
    db.js                optional / currently disabled if MongoDB is not used
  routes/
    plannerRoutes.js
    projectRoutes.js
  controllers/
    plannerController.js
    projectController.js
  engines/
    coverageEngine.js
    interferenceEngine.js
    healthEngine.js
    optimizerEngine.js
  data/
    projects/
      <Project_Name>.json
```

3. File-by-File Explanation

File	Role	What it does	Uses	Controls
server.js	Main Express entry point	Starts backend on port 5000, enables CORS, parses JSON, mounts planner and project routes.	Uses express, cors, dotenv, plannerRoutes, projectRoutes, optionally connectDB.	Controls whether frontend can reach /api/planner and /api/projects.
package.json	Backend dependency and script map	Lists installed backend packages and scripts like nodemon/server start.	express, cors, dotenv, nodemon, mongoose if kept.	Controls how backend is installed and started.
.env	Environment variables	Stores PORT and optionally MONGO_URI. Current JSON-file save system only needs PORT.	dotenv reads this file.	Controls backend port and optional database connection string.
config/db.js	MongoDB connection helper	Connects to MongoDB through mongoose. In your current JSON-file mode this can stay unused.	mongoose and process.env.MONGO_URI.	Controls database connection only if connectDB() is enabled in server.js.
routes/plannerRoutes.js	Planner API route table	Maps /api/planner/analyze and /api/planner/optimize	plannerController.js.	Controls which planner endpoints exist.

		to planner controller functions.		
routes/projectRoutes.js	Project API route table	Maps project save/load endpoints: POST /api/projects, GET /api/projects, GET /api/projects/:fileName.	projectController.js.	Controls project file listing and opening.
controllers/ plannerController.js	Planner request handler	Receives canvas payload from frontend, calls analysis/optimization engines, returns metrics and suggestions.	coverageEngine, interferenceEngine, healthEngine, optimizerEngine.	Controls analyze and optimize behavior returned to UI.
controllers/ projectController.js	Project save/load handler	Creates separate JSON file per project, lists available project files, reads a selected file.	fs, path, server/data/projects folder.	Controls persistent local project workspace.
engines/ coverageEngine.js	Coverage calculation brain	Divides canvas into grid cells and checks which cells are covered by infrastructure devices.	canvasWidth, canvasHeight, gridSize, nodes.	Controls coverage percent, dead-zone percent, heatmap cells.
engines/ interferenceEngine.js	Conflict detection brain	Checks overlapping device ranges and channel/frequency conflicts.	nodes with x, y, range, frequency, channel.	Controls red/amber/purple interference vectors and conflict count.
engines/healthEngine.js	Network health scorer	Starts from 100 and subtracts penalties for dead zones and interference.	coverage result + interference result.	Controls NET HEALTH INDEX.
engines/ optimizerEngine.js	Suggestion generator	Generates actions like change channel, move AP, add device near dead zone.	analysis data from coverage/interference/health.	Controls recommendation drawer suggestions.
data/projects/ <Project_Name>.json	Saved project file	Stores one complete AirGrid layout: project name, area blocks, nodes, visuals, last analysis.	projectController.js reads/writes it.	Controls loadable project state.

4. API Endpoints and Their Jobs

Endpoint	Triggered by	Backend response
POST /api/planner/analyze	Frontend sends current layout	Returns coverage, dead zones, interference, health score.
POST /api/planner/optimize	Frontend sends current layout	Returns analysis plus optimizer suggestions.
POST /api/projects	Save Project button	Creates one JSON file using the project name.
GET /api/projects	Load Projects button	Returns list of saved project files.
GET /api/projects/:fileName	Click a project in picker	Returns full JSON content so canvas can restore state.

5. Save and Load Project Flow

Save flow:

1. User clicks SAVE PROJECT. 2. Frontend asks for project name. 3. Frontend sends layout JSON to POST /api/projects. 4. projectController sanitizes the name and writes server/data/projects/<Project_Name>.json. 5. Backend returns success.

Load flow:

1. User clicks LOAD PROJECTS. 2. Frontend calls GET /api/projects. 3. Backend returns all available JSON filenames with summary metadata. 4. Project picker opens. 5. User clicks one file. 6. Frontend calls GET /api/projects/:fileName. 7. Canvas restores area size, nodes, visual toggles, and last analysis.

6. What Each Engine Should Calculate

Engine	Input	Output	Important Rule

coverageEngine	canvas size, grid size, nodes	cells, coveragePercent, deadZonePercent	Only WiFi AP / Router should solve WiFi coverage. BLE/IoT should not fake WiFi coverage.
interferenceEngine	nodes	interferenceVectors	Same frequency + same/near channel = stronger conflict.
healthEngine	coverage + interference	networkHealthScore	Score starts high, then loses points for dead zones and conflicts.
optimizerEngine	full analysis	suggestions[]	Recommend exact actions: add AP, move AP, change channel.

7. Health Index Calculation

Current concept: Health starts at 100, then penalties are subtracted. Dead zones reduce the score heavily. Critical same-channel interference reduces it further. Medium and low conflicts reduce it less.

Suggested simple formula:

networkHealthScore = 100 - deadZonePenalty - interferencePenalty

Example penalties: deadZonePenalty = deadZonePercent * 0.7; critical conflict = -15; medium conflict = -7; low conflict = -2.

8. Device Realism Rule

Device	Should Provide WiFi Coverage?	Should Affect	Reason
WiFi AP	Yes	Coverage, interference, health	Primary wireless coverage device.
Router	Yes, if treated as wireless/router combo	Coverage, backbone, health	In home/small network it can broadcast WiFi; in enterprise it may be core router.
BLE Beacon	No	Proximity / indoor positioning	Bluetooth beacon is not for internet coverage.
IoT Node	No	Load / sensor density / telemetry	Sensor consumes or sends data; it is not an access point.

9. Recommended Backend Improvements Next

- Move all visual calculations from frontend to backend so backend becomes the source of truth.
- Return heatmap cells from coverageEngine and render those cells in React-Konva.
- Add wall/obstacle support: walls should weaken signal and create realistic dead zones.
- Add delete/rename project file endpoints.
- Add PDF report export endpoint for coverage reports.
- Add validation so bad payloads cannot crash engines.

10. One-Line Summary

server.js receives the request, routes send it to controllers, controllers call engines or file storage, engines calculate network intelligence, and projectController saves/loads canvas layouts as JSON files.

Prepared for AirGrid MERN backend planning. JSON-file mode is suitable for local development and hackathon demos; MongoDB can be re-enabled later for production persistence.